

Interview Questions and Answers on UVM

1. Why do we use UVM?

UVM provides a standardized and reusable methodology for verifying complex designs. It helps build scalable testbenches using OOP concepts, reducing duplication and improving coverage and maintainability.

2. How is a UVM testbench different from a SystemVerilog testbench?

SystemVerilog gives us language constructs for verification, but UVM adds a structured framework with reusable components, sequences, and phases. UVM promotes modularity, reuse, and scalability.

3. Can you walk me through UVM phases?

Yes, UVM phases are ordered functions that manage the testbench lifecycle. Key ones include `build_phase`, `connect_phase`, `run_phase`, and so on. Each phase has a clear role—for instance, `build` is for construction, `connect` for TLM connections, and `run` for simulation behavior.

4. What is the UVM factory and why is it useful?

The factory allows dynamic creation of components and objects, supporting overrides without changing the base code. This is helpful for test customization and reuse.

5. How does the UVM Config DB work?

The Config DB is a global key-value store that lets you pass configuration data down the hierarchy without tight coupling between components.

6. What reporting mechanism does UVM offer?

UVM has built-in reporting with severities like `info`, `warning`, `error`, and `fatal`. We can control verbosity and customize the report handler for filtering or redirecting output.

7. What are TLM ports in UVM?

TLM ports are interfaces used to transfer transactions between components. They abstract signal-level interactions and allow higher-level modeling and communication.

8. What's the difference between new and create in UVM?

`new` is a standard SystemVerilog constructor, while `create` is a factory method that allows for overridden types and consistent object creation using factory rules.

9. What's the difference between a UVM object and a component?

Objects, like sequences and transactions, don't have phases. Components, like monitors or drivers, are phase-aware and part of the testbench hierarchy.

10. Can you write a basic UVM test class?

Yes:

```
class my_test extends uvm_test;
  `uvm_component_utils(my_test)
  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    // Create testbench components here
  endfunction
endclass
```

11. What are callbacks in UVM?

Callbacks allow behavior insertion at predefined hooks without changing the original component code. They're useful for injecting constraints or fault conditions.

12. What's the difference between an active and passive agent?

Active agents drive stimulus to the DUT using sequences and drivers. Passive agents just monitor DUT activity via monitors.

13. How does a UVM sequence get started?

We call `start()` on the sequence with a sequencer handle, which triggers the sequence's `body()` task to generate transactions.

14. How do you connect one monitor to multiple scoreboards?

Using analysis ports, a monitor can broadcast transactions. Multiple scoreboards can connect via analysis exports or FIFOs to receive the same data.

15. How do you connect multiple monitors to one scoreboard?

Each monitor has its own analysis port, and the scoreboard instantiates multiple analysis exports or FIFOs to collect data from all monitors.

16. How does a UVM testbench begin execution?

The `run_test()` function starts the simulation. It builds the test hierarchy, calls the phase methods, and drives the simulation.

17. What are UVM events used for?

UVM events help in synchronization between components or processes. They are useful in coordinating transaction generation and checking.

18. What is a UVM heartbeat?

Heartbeat is a monitoring mechanism to ensure the testbench is active. It prevents simulation hangs by checking for activity within a set interval.

19. What is a virtual sequence and virtual sequencer?

A virtual sequence controls multiple sequences running on different sequencers. A virtual sequencer aggregates handles to lower-level sequencers.

20. What's the role of `p_sequencer` and `m_sequencer`?

`p_sequencer` refers to the parent sequencer associated with a sequence. In layered setups, we sometimes use `m_sequencer` to refer to the master sequencer in virtual sequences.

21. What is the difference between config DB and resource DB?

Config DB is used primarily for configuration between components. Resource DB is a more general-purpose repository that can hold any type of shared resource.

22. How does sequence-driver handshaking work?

The driver calls `get_next_item` to get a transaction from the sequencer and `item_done` to complete it. This handshake ensures controlled transaction flow.

23. How do you implement basic scoreboard logic?

A scoreboard receives transactions from monitors and compares actual vs. expected values, logging errors if there's a mismatch.

24. What is the difference between a scoreboard and a subscriber?

A subscriber is a general-purpose analysis component. A scoreboard is a specialized subscriber used for result checking.

25. How does arbitration happen in sequencers?

When multiple sequences run simultaneously, the sequencer arbitrates which one provides the next item, typically in round-robin or priority-based fashion.

26. What are `uvm_utilities` and `uvm_macros` used for?

These provide macros like `uvm_component_utils` and `uvm_object_utils` that register components/objects with the factory and support utilities like printing and copying.

27. Can you describe the architecture of your project's UVM testbench?

Yes. It includes the environment containing agents (active/passive), monitors, drivers, sequencers, and scoreboards. Sequences are layered, and config DB is used to configure components. We also have virtual sequences to coordinate complex scenarios.

28. How do you perform coverage collection in UVM?

Functional coverage is typically added inside transaction classes using covergroups. These are sampled inside monitors or sequences to track test scenarios.

29. How do you use objections in UVM?

UVM uses objections (`raise_objection` and `drop_objection`) to coordinate phase completion. For example, in `run_phase`, tests raise an objection to keep the phase running until sequences complete.

30. How do you debug a UVM testbench?

I use UVM reports with increased verbosity, waveforms to trace transactions, print statements in sequences or monitors, and `uvm_info`/`uvm_error` to track flow. Factory overrides and verbose logs help isolate misbehaving components.

Notes by Pankaj